

Assignment No. 9

Subject: Database Engineering Lab (7CS271)

Class: SY CSE, AY 2025–26, Sem-II

Name - Atharv Vinayak Kulkarni

PRN - 245109074

Batch - S5

Indexing and Hashing

Q. Implement B+ Tree and Hash Indexing techniques using a programming language

Part A:

B+ Tree Implementation Implement a B+ Tree of order 'm'.
Support the following operations:

- Insertion of keys
- Searching for a key
- Traversal (leaf-level linked traversal)

Code :

part_a.cpp > ...

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  using namespace std;
5
6  #define MAX 3 // order of tree
7
8  class Node {
9  public:
10     bool isLeaf;
11     vector<int> keys;
12     vector<Node*> ptr;
13     Node* next;
14
15     Node(bool leaf) {
16         isLeaf = leaf;
17         next = NULL;
18     }
19 };
20
21 class BPlusTree {
22     Node* root;
23
24 public:
25     BPlusTree() {
26         root = new Node(true);
27     }
28
29     void insert(int key) {
30         Node* curr = root;
31
32         if (curr->keys.size() < MAX) {
33             curr->keys.push_back(key);
34             sort(curr->keys.begin(), curr->keys.end());
35         } else {
36             Node* newLeaf = new Node(true);
37             vector<int> temp = curr->keys;
38             temp.push_back(key);
39             sort(temp.begin(), temp.end());
40
41             curr->keys.clear();
42             for (int i = 0; i < (MAX+1)/2; i++)
43                 curr->keys.push_back(temp[i]);
44
45             for (int i = (MAX+1)/2; i < temp.size(); i++)
46                 newLeaf->keys.push_back(temp[i]);
```

```

47
48         newLeaf->next = curr->next;
49         curr->next = newLeaf;
50     }
51 }
52
53 void search(int key) {
54     Node* curr = root;
55
56     for (int k : curr->keys) {
57         if (k == key) {
58             cout << "Key found\n";
59             return;
60         }
61     }
62     cout << "Key not found\n";
63 }
64
65 void traverse() {
66     Node* curr = root;
67     while (curr != NULL) {
68         for (int k : curr->keys)
69             cout << k << " ";
70         curr = curr->next;
71     }
72     cout << endl;
73 }
74 };
75
76 int main() {
77     BPlusTree tree;
78
79     tree.insert(10);
80     tree.insert(20);
81     tree.insert(5);
82     tree.insert(15);
83
84     cout << "Traversal: ";
85     tree.traverse();
86
87     tree.search(15);
88     tree.search(100);
89
90     return 0;
91 }

```

Output:

```
Thu 16 Apr - 11:44 ~/Documents/WCE-Labs-Codes/DBEL/Assignment9 ' origin  main 4* 1
@atharv mkdir -p build && g++ part_a.cpp -o build/part_a && ./build/part_a
Traversal: 5 10 15 20
Key not found
Key not found
```

Part B:

Hash Indexing Implementation

Implement a hash table using: Division method: $h(k) = k \bmod n$ Handle collisions using: Chaining OR Open Addressing (Linear Probing)

Code:

```
part_b.cpp M X
part_b.cpp > HashTable > display()
...
1  #include <iostream>
2  using namespace std;
...
3  class HashTable {
4  private:
5      int *table;
6      int size;
7
8  public:
9      HashTable(int s) {
10         size = s;
11         table = new int[size];
12         for (int i = 0; i < size; i++)
13             table[i] = -1;
14     }
15
16     int hashFunction(int key) {
17         return key % size;
18     }
19
20     void insert(int key) {
21         int index = hashFunction(key);
22
23         // Linear probing
24         while (table[index] != -1) {
25             index = (index + 1) % size;
26         }
27
28         table[index] = key;
29         cout << "Inserted " << key << " at index " << index << endl;
30     }
31
32     void search(int key) {
33         int index = hashFunction(key);
34         int start = index;
35
36         while (table[index] != -1) {
37             if (table[index] == key) {
38                 cout << "Key " << key << " found at index " << index << endl;
39                 return;
40             }

```

```

41         index = (index + 1) % size;
42         if (index == start) break;
43     }
44
45     cout << "Key " << key << " not found" << endl;
46 }
47
48 void display() {
49     cout << "\nHash Table:\n";
50     for (int i = 0; i < size; i++) {
51         if (table[i] == -1)
52             cout << i << "- Empty \n";
53         else
54             cout << i << "- " << table[i] << endl;
55     }
56 }
57 };
58
59 int main() {
60     HashTable ht(10);
61
62     ht.insert(23);
63     ht.insert(33);
64     ht.insert(43); // collision case
65     ht.insert(13);
66
67     ht.display();
68
69     ht.search(33);
70     ht.search(99);
71
72     return 0;
73 }

```

Output:

```
Sun 19 Apr - 22:56 ~/Documents/WCE Lab Material/Database Engineering Lab/Assignment9 ' origin @ main 3
@atharv mkdir -p build && g++ part_b.cpp -o build/part_b && ./build/part_b
Inserted 23 at index 3
Inserted 33 at index 4
Inserted 43 at index 5
Inserted 13 at index 6

Hash Table:
0- Empty
1- Empty
2- Empty
3- 23
4- 33
5- 43
6- 13
7- Empty
8- Empty
9- Empty
Key 33 found at index 4
Key 99 not found
```